CUSTOMSDOC AI

Technical Design Document

AI-powered customs document extraction, cross-checking, human review, approval, and export

Project repository	https://github.com/jaikishan1234/customsdoc-ai
Application status	Local development only — not deployed
Demonstration record	SHP-24811 (synthetic data)
Production extraction model	gemini-3.5-flash-lite
Document date	9 November 2026

Prepared from the implemented application, source verification, and synthetic end-to-end demonstration. This document distinguishes implemented behavior from known limitations and future work.

1. Executive Summary

CustomsDoc AI is a web application intended to help customs documentation teams process shipment documents consistently. Users create a shipment, upload an invoice, packing list, and bill of lading (BOL), extract structured fields with a hosted vision-capable AI service, review uncertain or incorrect values, compare selected fields across documents, approve a verified shipment, and export the effective field values as CSV.

The implementation emphasizes traceability and controlled state transitions. Uploaded documents are versioned; extraction attempts are persisted; corrections are append-only; approvals capture an evidence snapshot; and export is gated by readiness and snapshot freshness. The demonstration uses synthetic documents only.

> Demonstrated outcome: shipment SHP-24811 completed extraction for all three document types. A Packing List gross weight of 3165 KG was corrected to 3120 KG to match the BOL. The discrepancy became obsolete, required fields were accepted, a manager approved the shipment, and the CSV export contained the corrected 3120 KG value with review status CORRECTED.

2. Problem and Objectives

Customs brokers and documentation teams work with multiple documents describing the same shipment. Values can be missing, formatted differently, or inconsistent between an invoice, packing list, and bill of lading. Manual comparison is repetitive, and an extraction result should not be treated as authoritative without review.

The system's objectives are to:

- Capture and version shipment documents in a controlled workflow.
- Extract catalog-defined structured fields and confidence scores from PDFs.
- Validate model output before persisting it.
- Compare selected cross-document values and surface discrepancies.
- Support manager-led correction, acceptance, approval, and CSV export.
- Preserve an audit trail and prevent stale approvals or post-export mutation.

3. Scope and User Roles

Role	Implemented capabilities
Documentation Executive	Create and view shipments; upload documents; initiate extraction; view extraction results and discrepancies. Cannot accept/correct fields, approve/reject, or export.
Documentation Manager	Executive capabilities plus field acceptance, append-only correction, approval/rejection, and CSV export.

The role is stored server-side on the user record. Sign-up does not accept a role from the client; new users default to DOCUMENTATION_EXECUTIVE. Privileged operations enforce the manager role on the server.

4. System Architecture

The application uses the Next.js App Router. Authenticated application pages are gated at the app layout, while Route Handlers expose shipment operations. Route handlers are intentionally thin: business rules reside in server-only service modules.

Layer	Technology / responsibility
Web application	Next.js 16 App Router, React 19, TypeScript, Tailwind CSS 4
Authentication	Better Auth email/password with Drizzle adapter and database-backed sessions
Application database	Neon PostgreSQL accessed through @neondatabase/serverless (neon-http); Drizzle ORM

Layer	Technology / responsibility
PDF object storage	Cloudinary authenticated raw assets; server-side signed delivery
AI extraction	Google Gemini Files API + Interactions API, plain server-side fetch; production model gemini-3.5-flash-lite
Business logic	Server-only services for shipments, documents, extraction, review, correction, discrepancy checks, approvals, invalidation, export, and audit
Deployment	Local development only; no production deployment completed

4.1 High-Level Request and Data Flow

The following is a logical view of the implemented components and their responsibilities.

Browser / React UI
Next.js App Router pages + Route Handlers
Better Auth session + server-side role gate
Server-only domain services
Neon PostgreSQL / Drizzle • Cloudinary storage • Gemini API

External services are accessed only from server-side modules. Cloudinary credentials and the Gemini API key are not exposed as public client variables. The database is the system of record for shipment state, document versions, extraction attempts, review decisions, corrections, approvals, and audit events.

5. Data Model

The database is organized around shipment lifecycle records, immutable document versions, extraction attempts, field-level review, discrepancies, approval evidence, and audit history. Better Auth tables support identity and sessions.

Table / group	Purpose
user, session, account, verification	Better Auth identity, credentials, sessions, and verification data.
shipments	Shipment reference, ownership/context, and lifecycle status.
documents	Document type, version, storage reference, checksum, active/superseded state, and upload metadata.
extractions	One persisted record per extraction attempt, status, and provider outcome.
extracted_fields	Catalog field values, confidence, page reference, and review status for an attempt.
field_corrections	Append-only corrected value snapshots, reason, actor, and timestamp.
discrepancies	Cross-document comparison result, severity, status, and evidence.
approvals	Decision and snapshot of document versions/checksums and extraction associations.
audit_logs	Append-only actor/system events and event details.

The implementation has one committed initial Drizzle migration containing the application and Better Auth tables. No schema changes were made during the later feature work described in this document.

6. Document Upload and Versioning

- Accepts PDF documents with a 25 MiB size limit, MIME and PDF magic-byte checks, and document-type validation.
- Computes a SHA-256 checksum and checks for duplicate content.
- Stores PDFs as authenticated raw Cloudinary assets; delivery is signed server-side rather than exposing public URLs.

- Each uploaded version is represented by a database row. A replacement becomes the active version and the prior version is retained as superseded history.
- The original filename is not reused as the Gemini provider display name; the provider upload uses a generic display name.

Storage is performed before the database record is finalized. Because the Neon HTTP driver does not provide a conventional interactive transaction boundary for this workflow, some multi-step operations have documented partial-failure windows; these are surfaced rather than represented as fully atomic.

7. AI Extraction Pipeline

The extraction pipeline processes the latest active version of each supported document type. Field catalogs define the expected output keys, types, and required fields for invoices, packing lists, and bills of lading.

Document type	Catalog fields
Invoice	9 fields, including invoice number/date, seller, buyer, currency, total amount, quantity, Incoterm, and payment terms.
Packing List	8 fields, including gross/net weight and package count.
Bill of Lading	13 catalog fields, including BOL identifiers/dates, container, vessel/voyage, ports, and shipment weight.

7.1 Provider interaction

- The server uploads the PDF through the Gemini Files API using a resumable upload flow.
- The provider file is polled until ACTIVE, with bounded polling and early handling of FAILED state.
- The Interactions API receives a document URI and a text prompt. Structured JSON output is requested using a schema generated from the field catalog.
- The production model is gemini-3.5-flash-lite with thinking_level set to low. Requests use store: false.
- The provider file is deleted in a finally block after the response is consumed; provider expiry is a fallback if cleanup fails.

The optional, text-only connectivity diagnostics endpoint is separate from the extraction pipeline and uses gemini-3.8-flash when explicitly enabled. It is not the production extraction model.

7.2 Validation and uncertainty

Provider output is treated as untrusted input. The server validates the response against the catalog: unknown keys invalidate the response; value types, date formats, lengths, and numeric precision are checked; confidence must be finite and within [0, 1]. Null or blank values are represented explicitly as absent fields. The system does not silently clamp invalid confidence values.

A confidence threshold of 0.90 is used to identify fields requiring human review. Extraction attempts are persisted with statuses. Failed or stale attempts do not become authoritative: the latest attempt number governs freshness, and a retry creates a new attempt rather than overwriting the prior record.

7.3 Failure handling and usage ceiling

Provider outcomes are typed (including invalid response, rejected, rate limited, daily ceiling reached, and unavailable). Timeouts and provider failures are converted to persisted failed attempts with error codes. Abandoned PROCESSING attempts older than the stale-processing threshold can be reaped on a subsequent run. A self-imposed daily request ceiling of 40 is enforced using an instance-local counter plus a durable database pre-check; the durable check-then-insert has a documented race window.

8. Cross-Document Discrepancy Detection

The implemented comparison is intentionally narrow: Packing List gross_weight_kg is compared with Bill of Lading gross_weight_kg when both evidence values are accepted or corrected and expressed in kilograms. The current tolerance is zero; any numeric inequality produces a CRITICAL discrepancy.

Condition	Resulting behavior
Comparable values differ	Create or maintain an OPEN discrepancy; severity CRITICAL.
Values become equal after correction	Mark the active discrepancy OBSOLETE; retain its history.

Condition	Resulting behavior
Evidence is missing or not current	Use RECHECK_PENDING until evidence can be evaluated.
Human RESOLVED / WAIVED state	Preserve the human decision; automated reevaluation does not overwrite it.

The successful demonstration began with a 45 KG mismatch (Packing List 3165 KG versus BOL 3120 KG). A manager corrected the Packing List to 3120 KG with a mandatory reason. The discrepancy was then marked OBSOLETE because the compared values matched.

9. Human Review and Correction

Managers can accept extracted fields or correct them. Acceptance is a guarded transition from PENDING_REVIEW to ACCEPTED and does not change the extracted value or confidence. Correction requires a valid catalog field, a mandatory reason, and a non-no-op value. Corrections are append-only; the original extraction remains intact and the latest correction is the effective value for downstream checks and export.

Field acceptance and correction are manager-only. The server verifies shipment/document ownership within the application context, active document version, latest completed extraction, and field validity before applying a change. Post-export mutation is blocked.

10. Application Screens and Demonstration

The screenshots below are from the local application and the synthetic SHP-24811 demonstration. They are included as evidence of implemented screens and workflow state, not as claims of a production deployment.

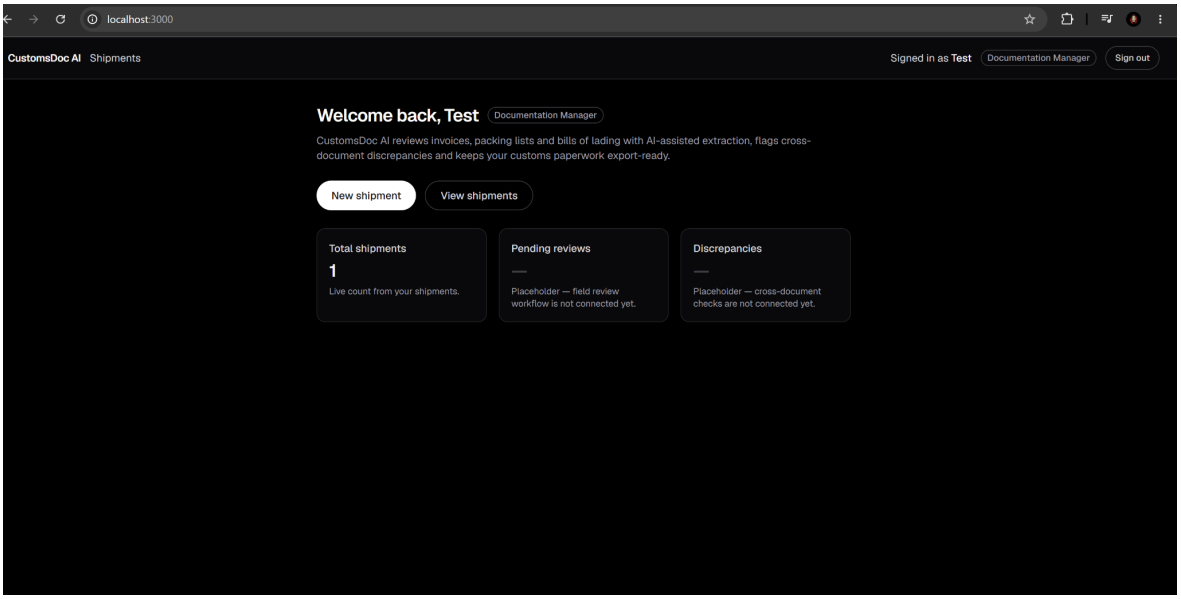


Figure 1. Dashboard. Total shipments is a live database count; Pending reviews and Discrepancies are labelled placeholders.

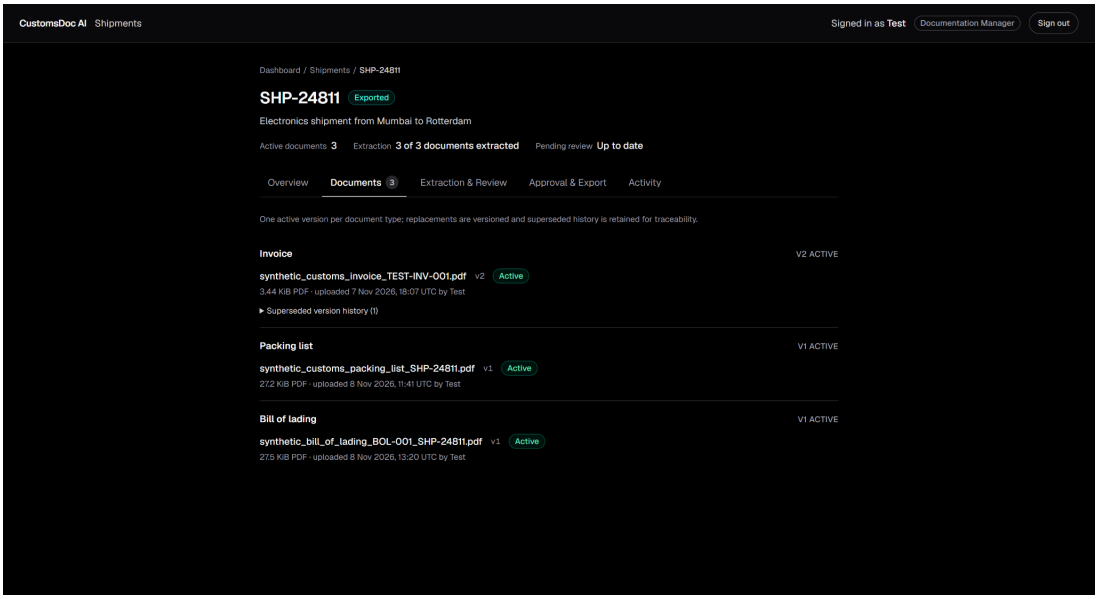


Figure 2. Document management showing Invoice v2 active, Packing List v1 active, BOL v1 active, and superseded invoice history.

10.1 Extraction and field review

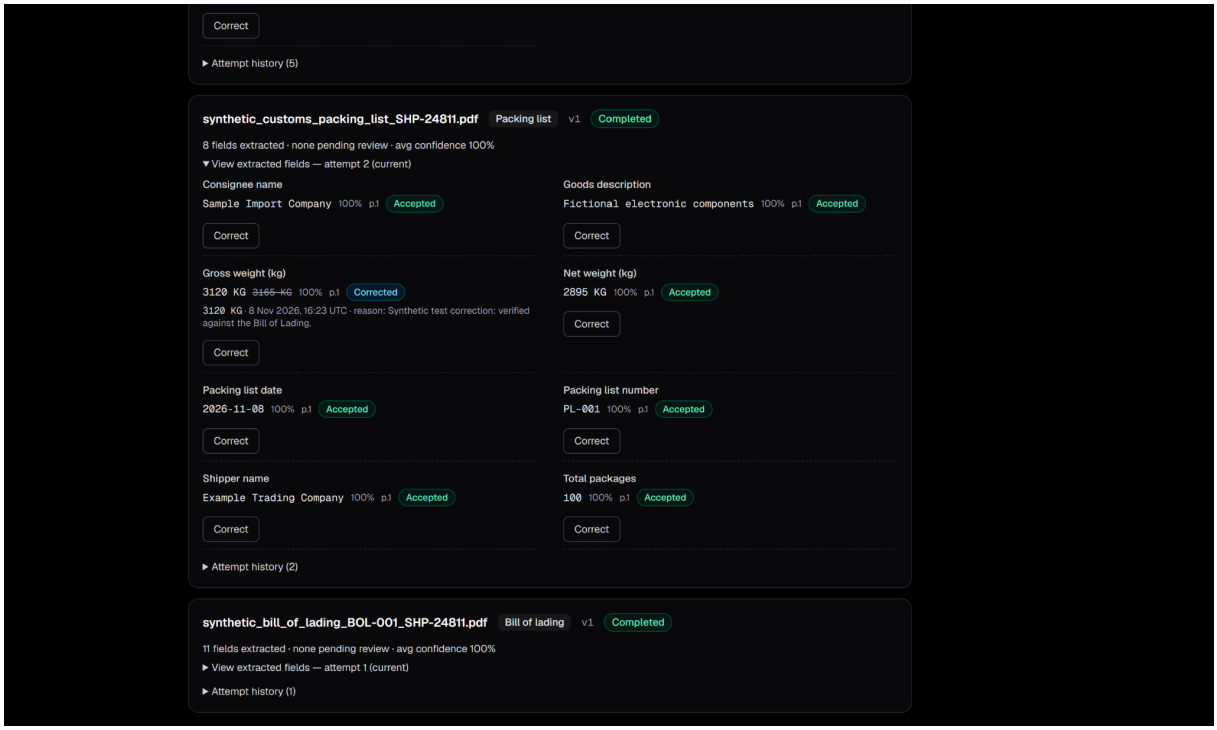


Figure 3. Packing List correction: gross weight changed from 3165 KG to 3120 KG; correction status and reason are visible.

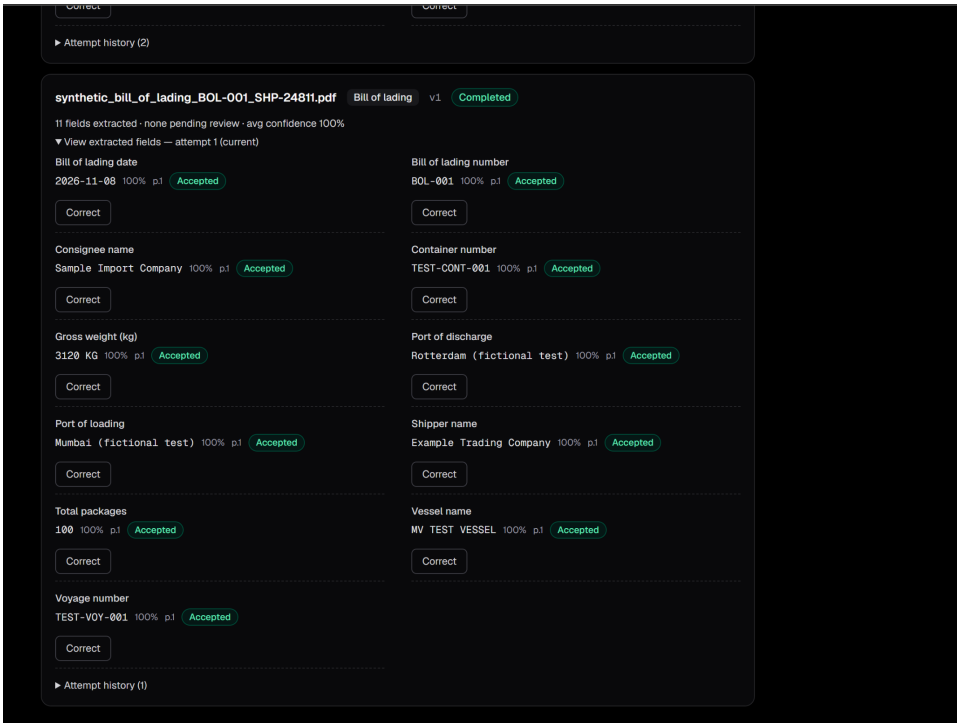


Figure 4. Bill of Lading extracted fields with accepted review status and 3120 KG gross weight.

10.2 Approval and export

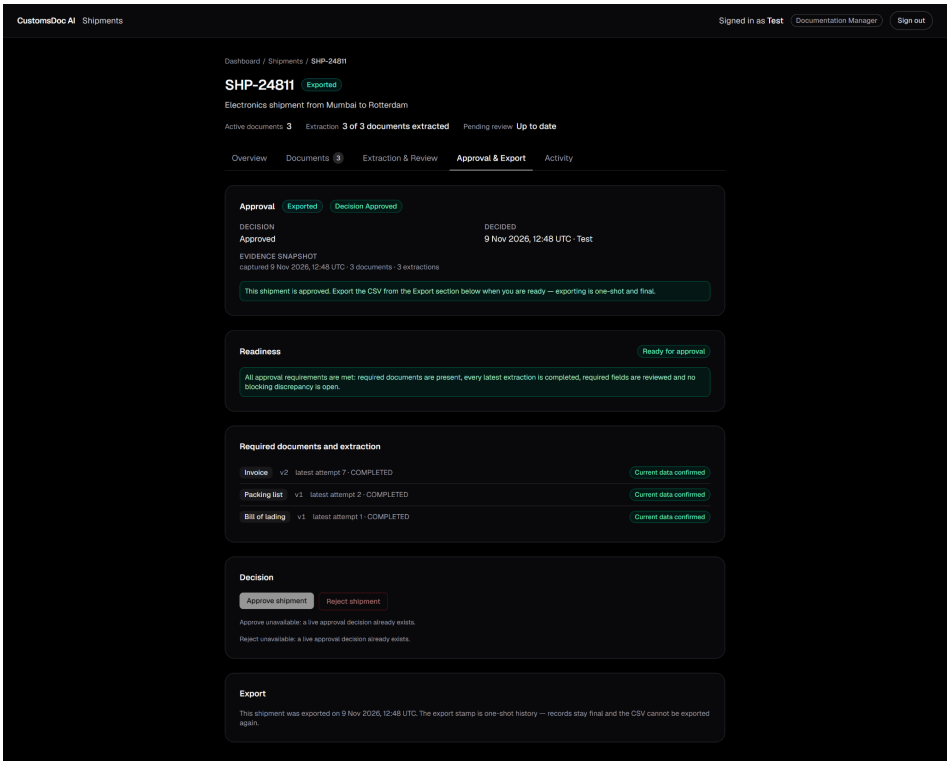


Figure 5. Approval and Export screen showing approved evidence snapshot and export state.

shipment_r	document_	document_	field_key	field_label	value	unit	confidence	review_status
SHP-24811	INVOICE	2	buyer_name	Buyer name	Sample Import Company		1	ACCEPTED
SHP-24811	INVOICE	2	currency	Currency	USD		1	ACCEPTED
SHP-24811	INVOICE	2	invoice_date	Invoice date	07-11-2026		1	ACCEPTED
SHP-24811	INVOICE	2	invoice_number	Invoice number	TEST-INV-001		1	ACCEPTED
SHP-24811	INVOICE	2	seller_name	Seller name	Example Trading Company		1	ACCEPTED
SHP-24811	INVOICE	2	total_amount	Invoice total amount	12500		1	ACCEPTED
SHP-24811	INVOICE	2	total_quantity	Total quantity	100		1	ACCEPTED
SHP-24811	PACKING_L	1	consignee_name	Consignee name	Sample Import Company		1	ACCEPTED
SHP-24811	PACKING_L	1	goods_description	Goods description	Fictional electronic components		1	ACCEPTED
SHP-24811	PACKING_L	1	gross_weight_kg	Gross weight (kg)	3120	KG	1	CORRECTED
SHP-24811	PACKING_L	1	net_weight_kg	Net weight (kg)	2895	KG	1	ACCEPTED
SHP-24811	PACKING_L	1	packing_list_date	Packing list date	08-11-2026		1	ACCEPTED
SHP-24811	PACKING_L	1	packing_list_number	Packing list number	PL-001		1	ACCEPTED
SHP-24811	PACKING_L	1	shipper_name	Shipper name	Example Trading Company		1	ACCEPTED
SHP-24811	PACKING_L	1	total_packages	Total packages	100		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	bol_date	Bill of lading date	08-11-2026		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	bol_number	Bill of lading number	BOL-001		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	consignee_name	Consignee name	Sample Import Company		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	container_number	Container number	TEST-CONT-001		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	gross_weight_kg	Gross weight (kg)	3120	KG	1	ACCEPTED
SHP-24811	BILL_OF_LA	1	port_of_discharge	Port of discharge	Rotterdam (fictional test)		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	port_of_loading	Port of loading	Mumbai (fictional test)		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	shipper_name	Shipper name	Example Trading Company		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	total_packages	Total packages	100		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	vessel_name	Vessel name	MV TEST VESSEL		1	ACCEPTED
SHP-24811	BILL_OF_LA	1	voyage_number	Voyage number	TEST-VOY-001		1	ACCEPTED

Figure 6. Exported CSV opened in Excel. The effective Packing List gross weight is 3120 KG and marked CORRECTED.

11. Approval and Snapshot Freshness

Approval readiness is evaluated server-side. The system checks that all three required document types are present and active, latest extraction attempts are completed, required fields are accepted or corrected, and no blocking OPEN or RECHECK_PENDING discrepancies remain. A manager can approve or reject only after the readiness evaluation passes.

On approval, the service stores a snapshot containing active document identifiers, types, versions, checksums, extraction associations, and capture time. Subsequent material changes—such as a new document version, new extraction attempt, field correction, or a blocking discrepancy—can invalidate the approval and return the shipment to REVIEW. The approval UI also checks snapshot freshness before export.

12. CSV Export

Export is manager-only and requires an approved shipment, a current approval snapshot, required documents and completed latest attempts, reviewed required fields, and no blocking discrepancies. A guarded one-shot state transition stamps the shipment EXPORTED with export time and actor. Exported shipments are locked against further upload, extraction, correction, and acceptance.

The CSV includes shipment reference, document type/version, field key/label, effective value, unit, confidence, and review status. Rows are deterministic; output uses UTF-8 and RFC 4180-style CRLF records with quoting where required. Formula-injection protection prefixes values beginning with =, +, -, or @ (and selected control prefixes). Corrected fields export the latest effective correction, not the original extracted value.

A known operational limitation is that a failure after the EXPORTED stamp but before the browser completes the download can leave the shipment exported without the user receiving the file. The UI advises refresh rather than a blind retry; there is no separate export recovery workflow.

13. Audit Trail

Audit events record key shipment actions, including creation, document upload, extraction, discrepancy reevaluation, field correction/acceptance, approval decisions and invalidation, export, and recovery markers. Audit logging is append-oriented and uses a single sanctioned writer. Audit failures are surfaced as warnings rather than silently ignored. An opt-in, role-gated diagnostics route can report audit gaps without exposing document contents, storage keys, or checksums.

14. Security, Privacy, and Operational Controls

- Server-side authentication and role checks protect pages and privileged operations.
- Roles are server-owned; client-supplied role escalation is not accepted.
- Cloudinary assets are authenticated and delivered through signed server-side access.
- Gemini calls are server-side; API credentials are not exposed to the browser; generation uses store: false.
- Only synthetic documents were used for the demonstration. No real customs/customer documents are included.
- PDF size, MIME type, magic bytes, field schema, response size, and numeric precision are validated.
- Diagnostics are disabled by default and return 404 unless explicitly enabled.
- Secrets belong in local environment configuration and are not included in source control or this document.

15. Verification and Demonstration Results

Area	Demonstration / verification
Document upload	Synthetic Invoice v2, Packing List v1, and BOL v1 active for SHP-24811.
AI extraction	Latest attempts completed: Invoice #7, Packing List #2, BOL #1.
Field review	Eight required fields across Invoice and BOL were accepted after review.
Correction	Packing List gross weight corrected from 3165 KG to 3120 KG; original preserved.
Discrepancy	45 KG mismatch became OBSOLETE after correction to matching 3120 KG.
Approval	Manager approval succeeded and captured a snapshot for 3 documents and 3 extraction attempts.
Export	CSV downloaded and inspected in Excel; corrected 3120 KG value present with CORRECTED status.

Verification was manual using synthetic data and the application UI/API workflow. There is no automated test suite or test script in the repository; this document does not claim automated test coverage.

16. Known Limitations

- The application has not been deployed; it runs locally only. No live application URL is available.
- No automated test suite is present. Verification described here is manual.
- Only Total shipments is a live dashboard metric. Pending reviews and Discrepancies are placeholders and are not connected to their respective dashboards.
- Cross-document comparison currently covers only Packing List versus BOL gross weight in KG, with zero tolerance.
- The 40-request daily ceiling is best-effort and the durable check-then-insert has a race window.
- The neon-http workflow has documented partial-failure windows where operations span multiple statements.
- There is no per-shipment user authorization boundary beyond the authenticated application and role checks.
- CSV is the implemented export format; there is no PDF/report export.
- Correction, acceptance, approval, and export are manager-only; an executive cannot complete a shipment alone.
- The CSV formula guard can prefix legitimate negative-number values.

17. Future Improvements

- Deploy to a production hosting environment and document the production URL, monitoring, and operational configuration.
- Add automated unit, integration, and end-to-end tests for validation, role gates, state transitions, approval invalidation, and export concurrency.
- Expand discrepancy rules to cover additional fields, units, tolerances, and document combinations.
- Implement connected dashboard summaries for pending reviews and discrepancies.
- Add robust recovery for export failures after the shipment state transition.
- Improve transaction/compensation guarantees for multi-step persistence workflows.
- Introduce per-shipment authorization and finer-grained operational permissions.
- Add broader observability and configurable rate limiting.

18. Local Setup Reference

The repository's deployment.md is the authoritative local setup guide. It documents Node.js prerequisites, npm commands, environment variable names, database migration steps, Better Auth, Cloudinary, Gemini configuration, and demo-account setup. The application is not deployed; the Vercel section is a future deployment plan only.

Environment variable names documented by the project: DATABASE_URL, BETTER_AUTH_SECRET, BETTER_AUTH_URL, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, GEMINI_API_KEY. Optional diagnostics flags: GEMINI_DIAGNOSTIC_ENABLED and AUDIT_DIAGNOSTIC_ENABLED. No secret values are included in this document.

19. Conclusion

CustomsDoc AI implements an end-to-end customs document workflow with structured AI extraction, server-side validation, human review, append-only corrections, cross-document checking, evidence-based approval, and guarded CSV export. The SHP-24811 synthetic demonstration exercised the principal workflow and confirmed that the corrected value flowed through discrepancy reevaluation, approval, and export. The document also records the current limits honestly: local-only deployment, no automated test suite, narrow discrepancy scope, and unfinished dashboard summaries.

Repository: https://github.com/jaikishan1234/customsdoc-ai

End of document